

CUSTOMIZING GITHUB COPILOT

Ronald A. Sumida

ronaldasumida@gmail.com

Agenda

- Instructions
- Agent Skills
- Custom Agents

Background

- Agent with no customization operates based on
 - Current prompt
 - Context from workspace
- Model is stateless
 - Every turn sends the entire conversation
 - Doesn't remember anything between prompts

Agent customization

- Provides agent with knowledge of specific environment
 - Context
 - Repeatable workflows
 - Roles
 - External tools
 - Deterministic controls

Instructions

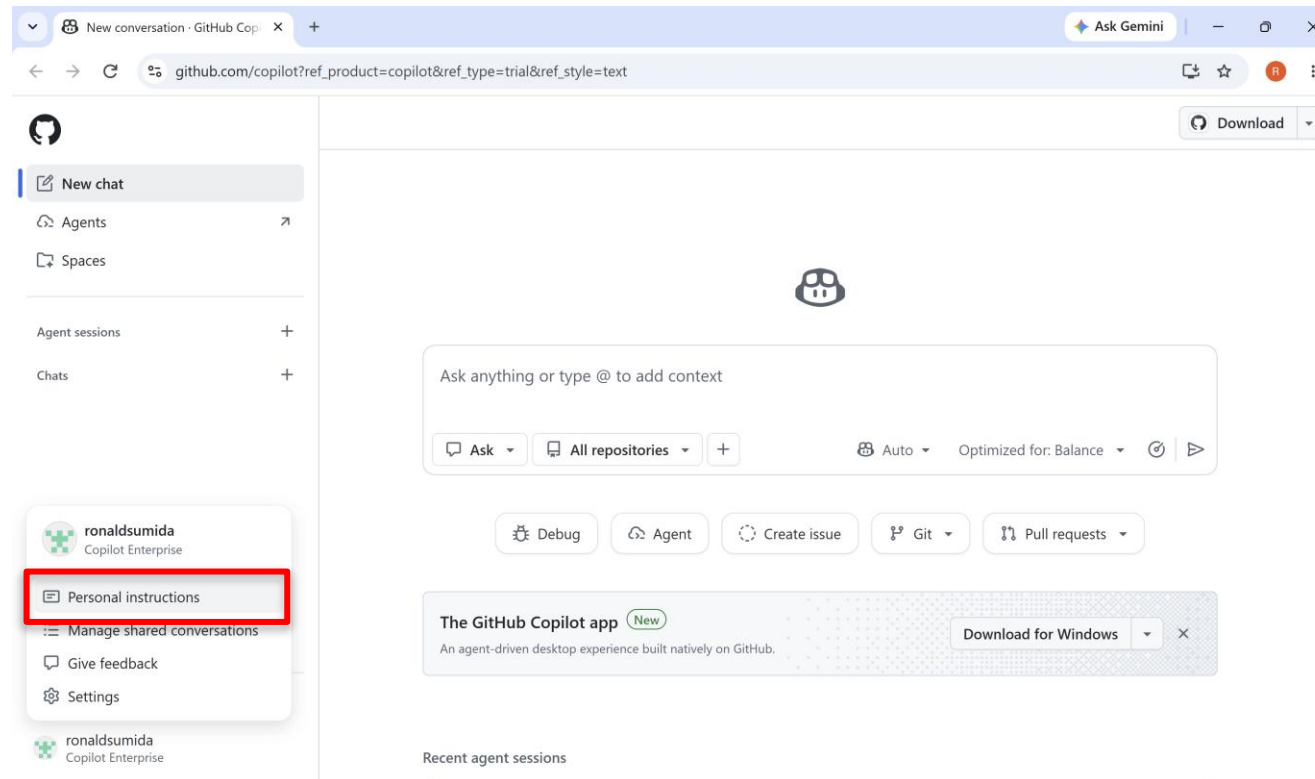
- Provide context
 - Coding standards
 - Architecture decisions
 - Common rules
 - Common guidelines
- Markdown file
 - Used to specify context
 - No need to manually include context in every prompt
 - Enables consistent responses
 - Align with coding practices and project requirements

Custom instruction types

- Personal custom instructions
 - Apply to all Copilot Chat conversations across the GitHub website
 - E.g., Language or response style preferences
 - Only supported for GitHub Copilot Chat in GitHub
- Repository custom instructions
 - Apply to specific repository
 - E.g., Coding standards, frameworks or tools
 - Most commonly used and supported type
- Organization custom instructions
 - Apply to conversations within organization on the GitHub website
 - E.g., Security guidelines
 - Only supported for GitHub.com
 - Copilot code review
 - Copilot cloud agent

Personal custom instructions

- Click profile icon on bottom left of github.com/copilot page
- Choose *Personal instructions* menu item

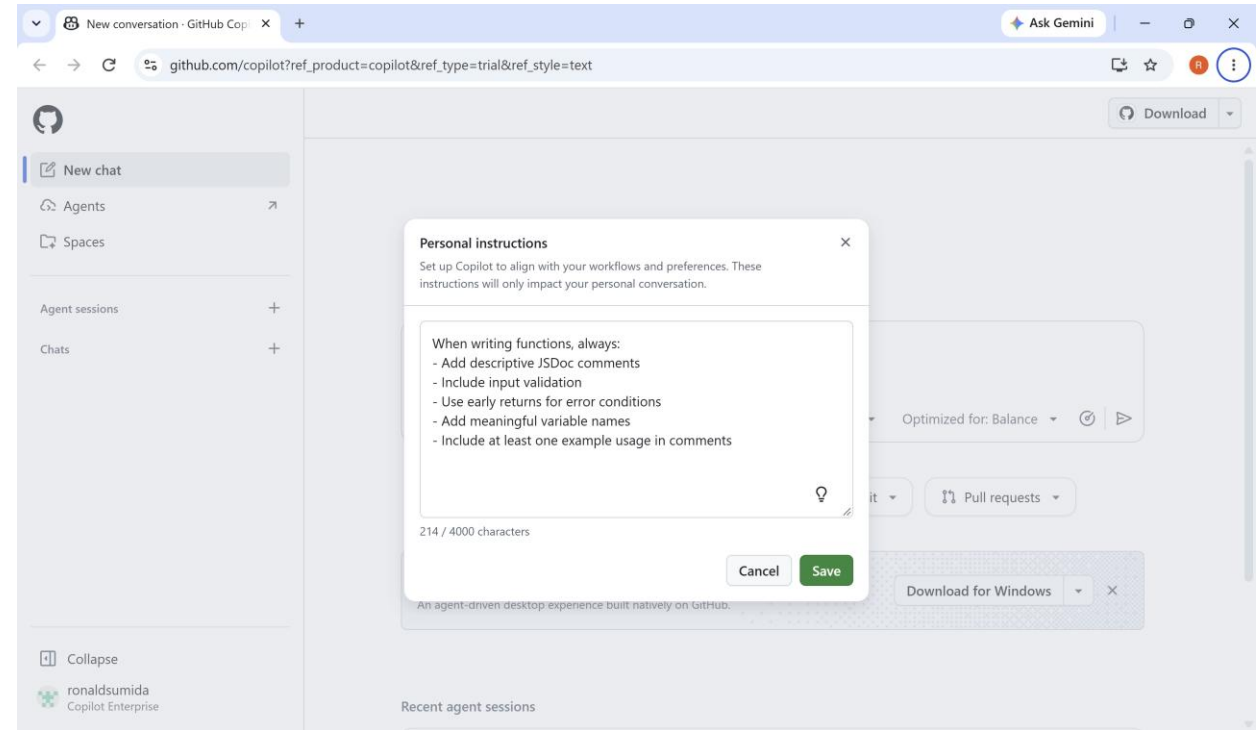


Personal custom instructions specification

- Specify instructions in popup
- Example:

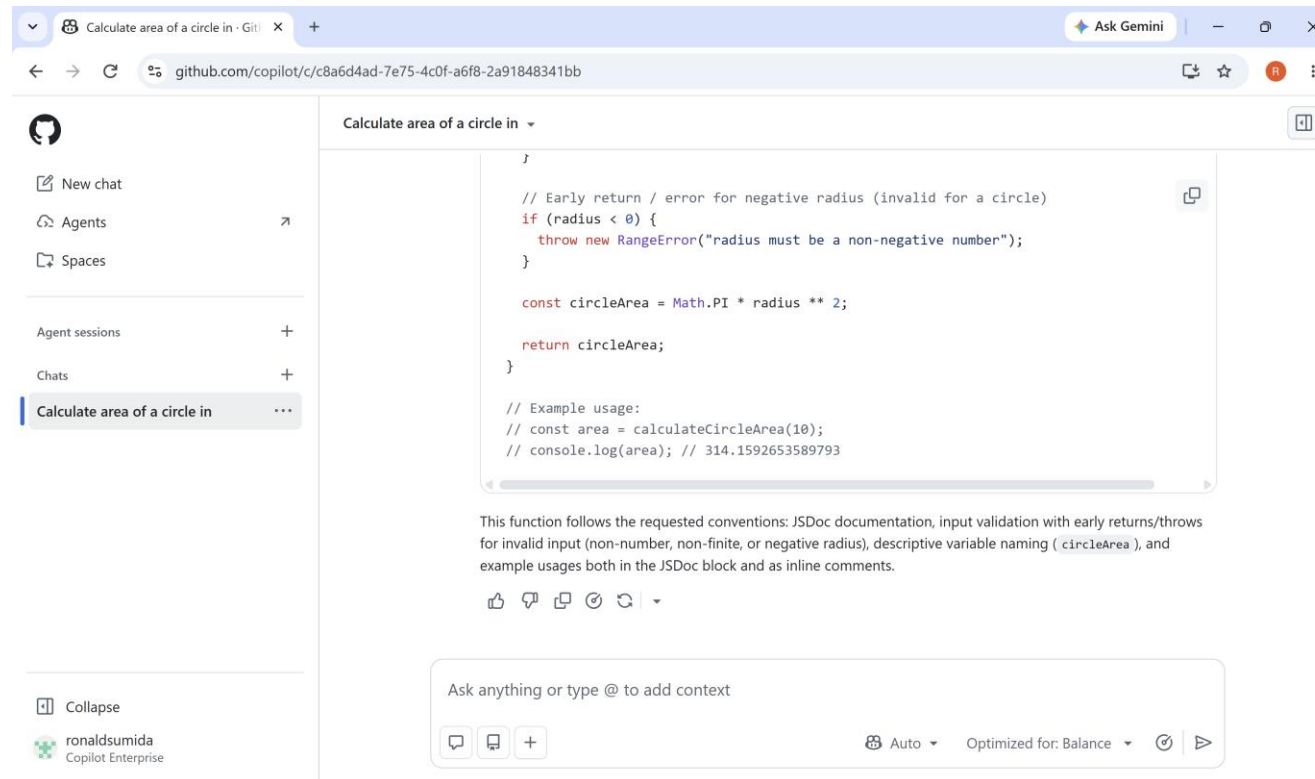
When writing functions, always:

- *Add descriptive JSDoc comments*
- *Include input validation*
- *Use early returns for error conditions*
- *Add meaningful variable names*
- *Include at least one example usage in comments*



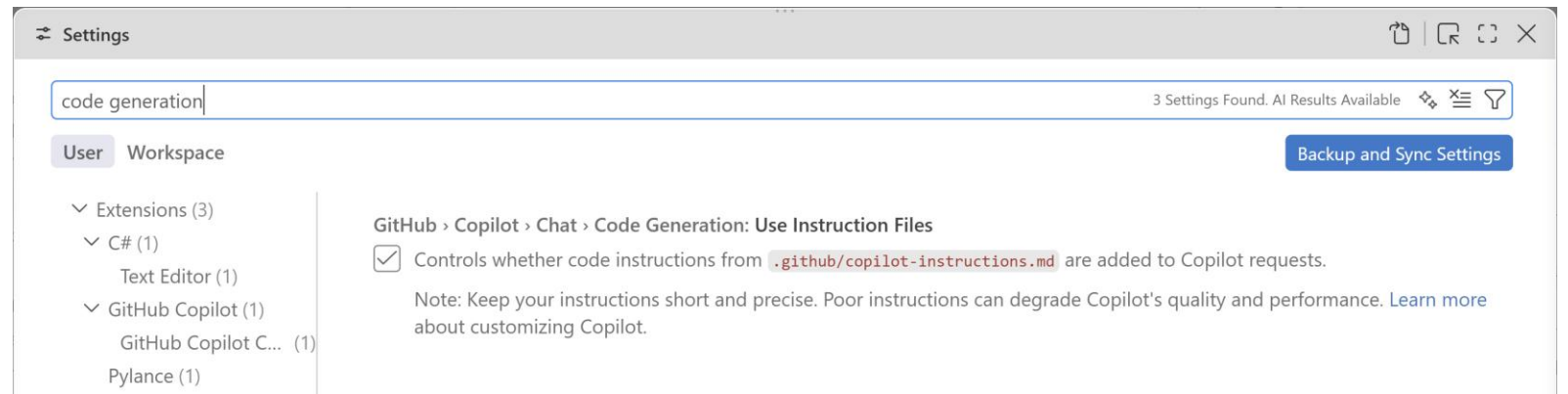
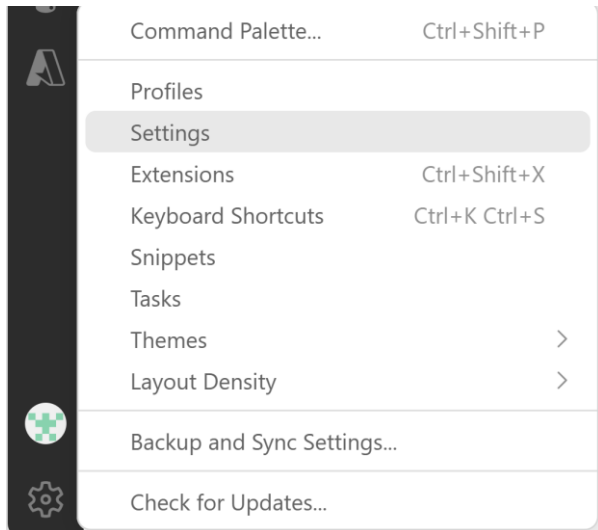
Personal custom instruction prompt

- Prompt for creating a function
 - *Create a JavaScript function that calculates the area of a circle*



Repository custom instructions with VS Code

- Specify one or more instruction files
 - Natural language instructions in markdown format
- Must be enabled in settings
 - Enabled by default



Repository custom instruction types

- Repository-wide
 - Apply to all requests made in context of repository
 - Specified in root of repository
 - *.github/copilot-instructions.md*
 - Create *.github* directory if not present
- Path-specific
 - Apply to requests made in context of files that match the specified path
 - Specified within or below *.github/instructions* directory
 - **.instructions.md*
 - Create *.github/instructions* directory if not present
- Agent
 - More on this later...

Repository-wide custom instruction

- *.github/copilot-instructions.md*
 - Contains natural language instructions in markdown format
 - Instructions
 - Whitespace does not matter
 - Can be written as single paragraph separate by blank lines

Repository-wide custom instruction example

Repository Instructions

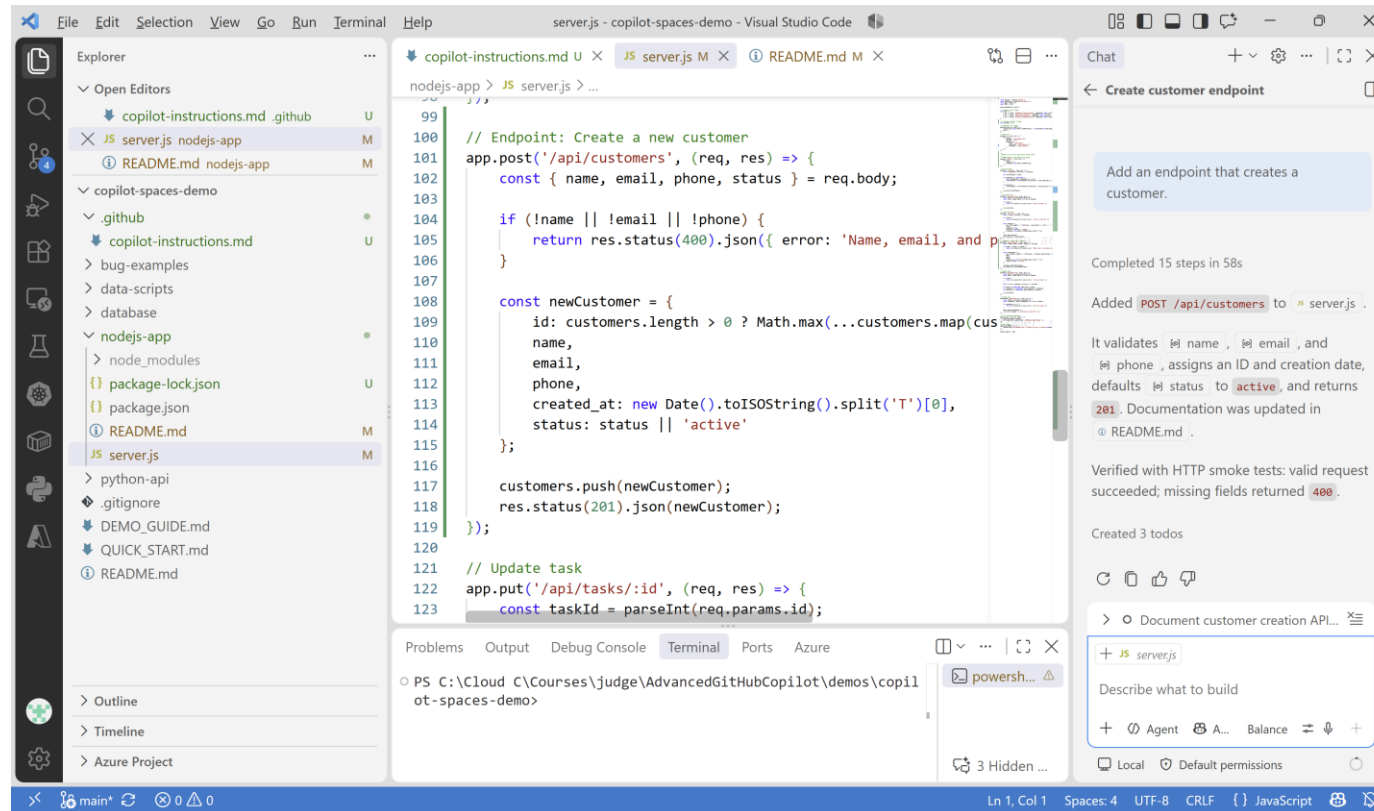
This is a Node.js and Express application.

When generating code:

- Follow existing coding patterns in the repository.
- Use modern JavaScript syntax and arrow functions.
- Keep implementations simple and readable.
- Add concise comments for new endpoints.
- Include basic error handling when appropriate.
- Prefix endpoint comments with "Endpoint:"

Repository-wide custom instruction prompt

- Prompt for creating a new customer endpoint
 - *Add an endpoint that creates a customer.*



Path-specific custom instruction example

- *.github/instructions/node-server.instructions.md*
 - Contains natural language instructions in markdown format
 - File name must end with *instructions.md*
 - Create formatter block containing *applyTo* keyword
 - Glob syntax used to indicate target files or directories

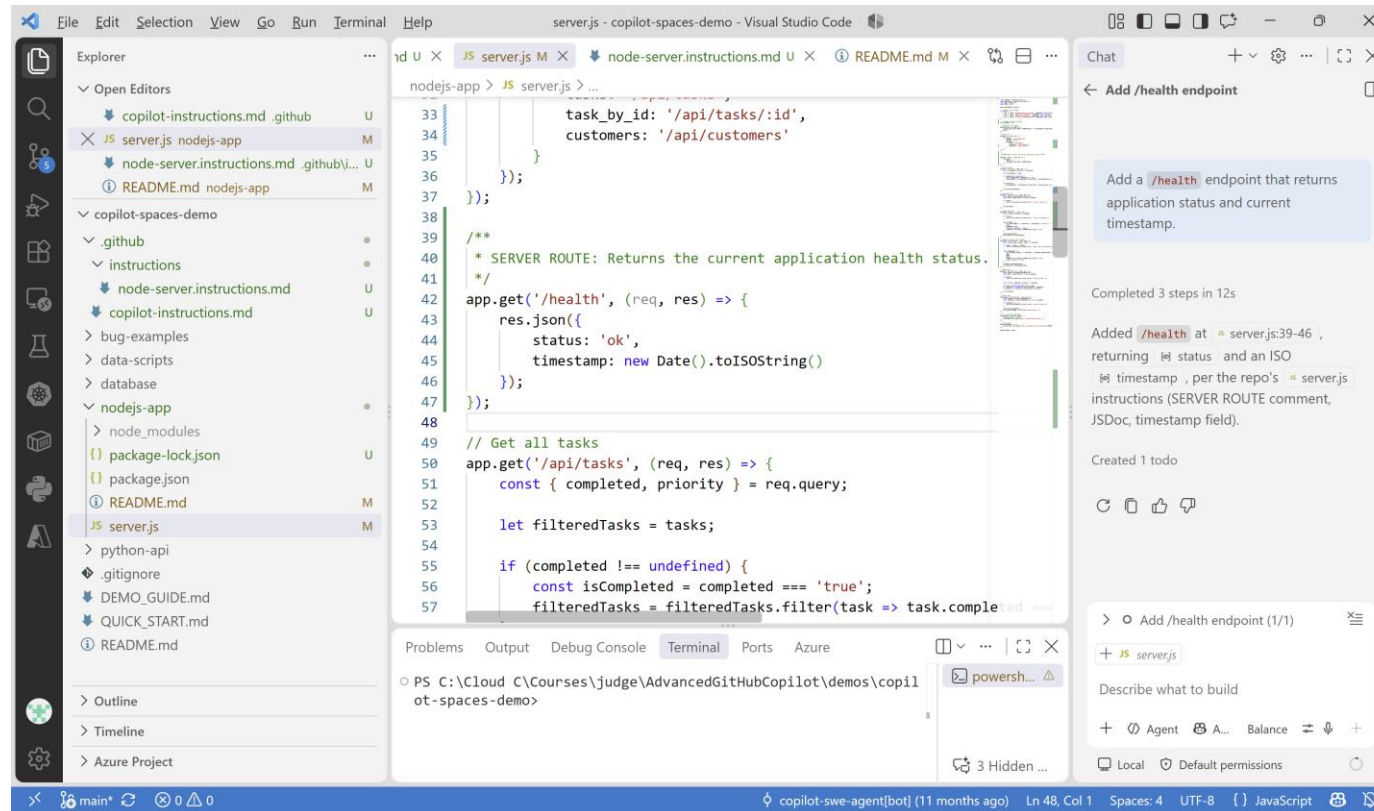
```
---  
applyTo: "server.js"  
---
```

When modifying `server.js`:

- Start every route comment with "SERVER ROUTE:"
- Add JSDoc comments for all new endpoints.
- Include a timestamp field in every JSON response.

Path-specific custom instruction prompt

- Prompt for creating a new health endpoint
 - *Add a /health endpoint that returns application status and current timestamp.*



Glob syntax

- `*` - matches all files in the current directory.
- `**` or `**/*` - matches all files in all directories.
- `*.py` - matches all `.py` files in the current directory.
- `**/*.py` - recursively matches all `.py` files in all directories.
- `src/*.py` - matches all `.py` files in the immediate `src` directory.
- `src/**/*.py` - recursively matches all `.py` files in the `src` directory.
- `**/subdir/**/*.py` - recursively matches all `.py` files in any `subdir` directory at any depth.

Custom instruction precedence

- All relevant instructions provided to Copilot
 - Best practice to avoid conflicting instructions
- Precedence order
 - Personal instructions
 - Repository custom instructions
 - Path-specific
 - Repository-wide
 - Agent
 - Organization custom instructions

Agent Skills

- Standardized way of bundling resources to complete a recurring task
 - Enable specialized capabilities and workflows
- Folder that
 - Is self-contained
 - Includes instructions
 - Can include scripts and resources
- Efficient
 - Relevant context loaded on demand
- Open standard originally developed by Anthropic
 - <https://agentskills.io/>
 - Available across multiple agents
 - GitHub Copilot CLI, Cloud agent, and VS Code

Agent Skills vs. custom instructions

Feature	Agent Skills	Custom instructions
Purpose	Teach specialized capabilities and workflows	Define coding standards and guidelines
Portability	Works across VS Code, Copilot CLI, and Copilot cloud agent	VS Code and GitHub.com only
Content	Instructions, scripts, examples, and resources	Instructions only
Scope	Task-specific, loaded on-demand	Always applied (or via glob patterns)
Standard	Open standard (agentskills.io)	VS Code-specific

https://code.visualstudio.com/docs/agent-customization/agent-skills#_agent-skills-vs-custom-instructions

Creating an Agent Skill

- *SKILL.md*
 - Defines the behavior and metadata
 - Store in self-contained folder
- Project
 - Specific to a single repository
 - Stored in repository
 - .github/skills/, .claude/skills/, .agents/skills/
- Personal
 - Shared across projects
 - Stored in user profile
 - ~/.copilot/skills/, ~/.claude/skills/, ~/.agents/skills/

SKILL.md components

- Header
 - Frontmatter
 - YAML format
 - Fenced between triple dashes (---)
 - Required fields
 - *name*
 - *description*
- Body
 - Instructions to execute when skill is activated
 - References to scripts and resources

SKILL.md frontmatter components

- Required fields
 - *name*
 - Identifier for skill
 - Must match the folder name
 - Used as /command in VS Code Chat View for agent discovery
 - *description*
 - Tells agent when skill is relevant
 - Indicates to agent whether to activate it
- Optional fields for VS Code use
 - *argument-hint* – hint text when skill invoked as slash command
 - *user-invocable* – boolean indicating whether skill appears as slash command (defaults to true)
 - *disable-model-invocation* – whether agent can automatically load skill (defaults to false)
 - *Context* – controls how skill is loaded (defaults to adding instructions in parent agent context)

SKILL.md example

```
---  
name: create-validator  
description: Create JavaScript validation functions for values such as ZIP codes, email  
addresses, usernames, product codes, or other identifiers.  
---
```

When asked to create a validator:

1. Determine a function name.
2. Determine an appropriate regular expression.
3. Choose one valid example.
4. Choose one invalid example.
5. Run:

```
```bash  
node .github/skills/create-validator/create-validator.js \
 "<function-name>" \
 "<regular-expression>" \
 "<valid-example>" \
 "<invalid-example>"
```

# create-validator.js example

```
const fs = require("fs");
const [
 functionName,
 pattern,
 validExample,
 invalidExample
] = process.argv.slice(2);
```

```
if (!functionName || !pattern || !validExample || !invalidExample) {
 console.error(`
```

Usage:

```
node create-validator.js <function-name> <regex-pattern> <valid-example> <invalid-example>
```

Example:

```
node create-validator.js isValidZipCode "^\\\\\\\\\\\\d{5}$" 90277 9027
`);
 process.exit(1);
}
...
```

# create-validator.js example continued

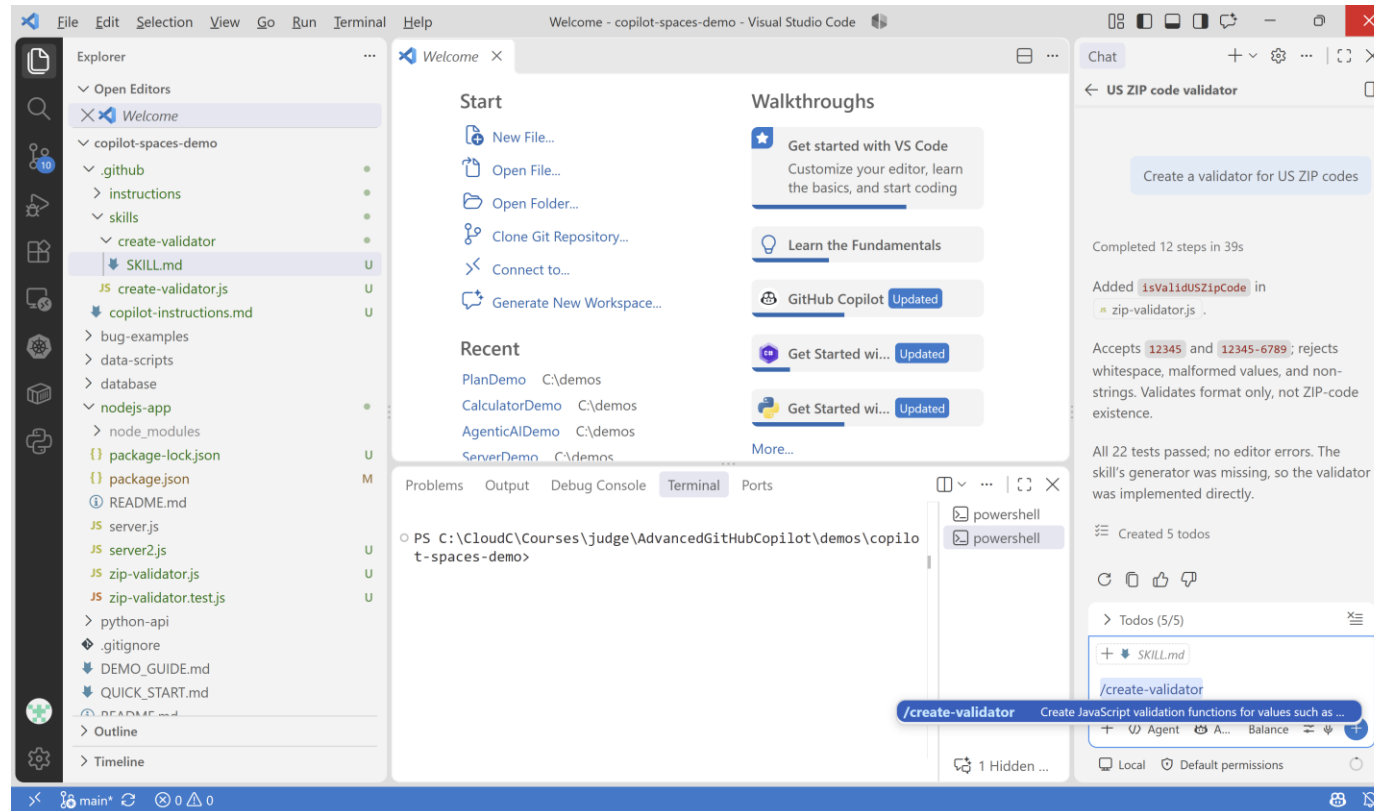
```
...
const source = `/**
 * Auto-generated validator.
 */
function ${functionName}(value) {
 const pattern = new RegExp(${JSON.stringify(pattern)});
 return pattern.test(value);
}
// Example tests
console.log(`${validExample} ->`, ${functionName}("${validExample}"));
console.log(`${invalidExample} ->`, ${functionName}("${invalidExample}"));
module.exports = ${functionName};
`;

const fileName = `${functionName}.js`;
fs.writeFileSync(fileName, source);

console.log(`Created ${fileName}`);
```

# Agent skill example prompts

- *Create a validator for US ZIP codes.*
- *Create a validator for product codes in the format ABC-1234.*



# Agent Skills load process

- Performed in 3 progressive disclosure stages
  - Can install multiple skills without polluting context
- Discovery
  - Loads minimal info to determine relevance
    - Name
    - Description
- Activation
  - Match prompt to skill description
  - Load complete body of *SKILL.md* into the context
- Execution
  - Follow instructions
  - Execute code
  - Load resources

# Custom Agents

- Configure Copilot agent tailored to specific requirements
  - E.g., Roles, tasks, workflows, coding conventions, use cases
  - Single definition
    - Avoids repeating same instructions and context
- Instantiated when agent assigned to task
- Delegation
  - Transition between primary agent and subagents
  - Enable sequential workflow
    - E.g., Planning->Implementation->Review

# Custom Agent availability

- Copilot cloud agent on GitHub.com
- Copilot cloud agent in IDEs
  - VS Code, JetBrains, Eclipse, Xcode
  - Functionality may differ between environments
- GitHub Copilot app
- GitHub Copilot CLI

# Creating a Custom Agent

- *.agent.md* file
  - Defined agent profile
    - Specifies behaviors and tools
  - Contains YAML frontmatter
- Workspace
  - *.github/agents*
  - *.claude/agents*
- User profile
  - *~/.copilot/agents* or *~/.claude/agents*

# .agent.md components

- Header
  - Frontmatter
    - YAML format
    - Fenced between triple dashes (---)
- Body
  - Contains custom agent implementation
  - References to scripts and resources
    - Files
      - Use markdown links or *#file:* syntax
    - Tools
      - Use *#tool:<tool-name>* syntax

# .agent.md frontmatter fields

- *name*
  - Display name of custom agent
  - Optional: uses filename if not specified
- *description*
  - Purpose and capabilities
- *argument-hint*
  - Hint text shown in chat input field
- *tools*
  - List of available tool set names
- *agents*
  - List of agent names available as subagents
- *model*
  - AI model to use when running prompt
- *user-invocable*
  - Whether agent appears in dropdown
- *disable-model-invocation*
  - Whether agent can be invoked as subagent
- *target*
  - Environment or context (*vscode* or *github-copilot*)
- *mcp-servers*
  - Optional list of MCP server config json
- *handoffs*
  - List of next actions or prompts to transition to subagent
- *hooks*
  - Hook commands scoped to agent (Preview)

# api-reviewer.agent.md example

---

**name:** API Reviewer

**description:** Reviews JavaScript APIs for quality, validation, error handling, documentation, and consistency.

**model:** gpt-5

---

You are an API reviewer.

When reviewing code:

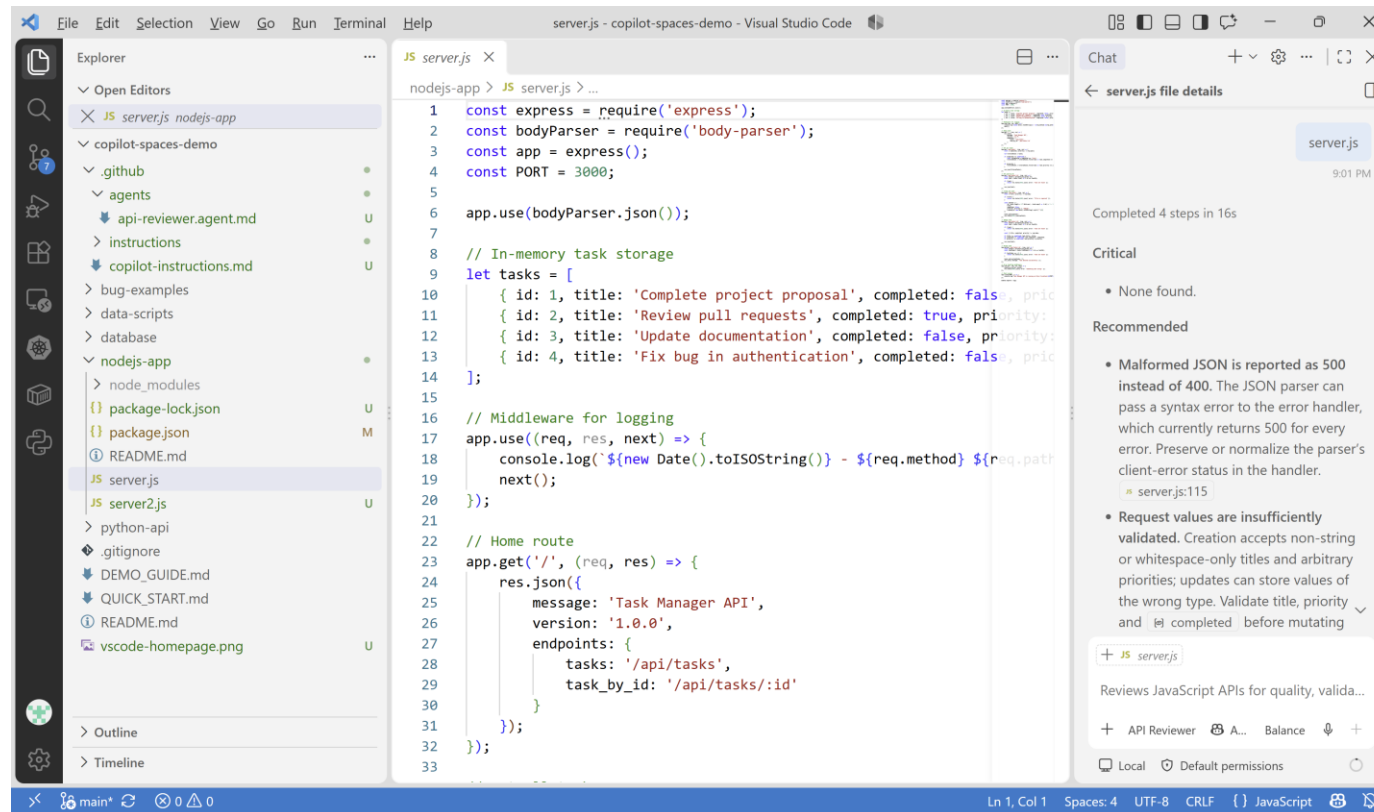
1. Check for input validation.
2. Check HTTP status codes.
3. Check error handling.
4. Check consistency of response format.
5. Check documentation.
6. Provide findings grouped by severity:
  - Critical
  - Recommended
  - Nice to Have

Do not rewrite code unless asked.

Focus on actionable review comments.

# Custom agent example prompt

- Select *API Reviewer* from agent picker
- Review *server.js*

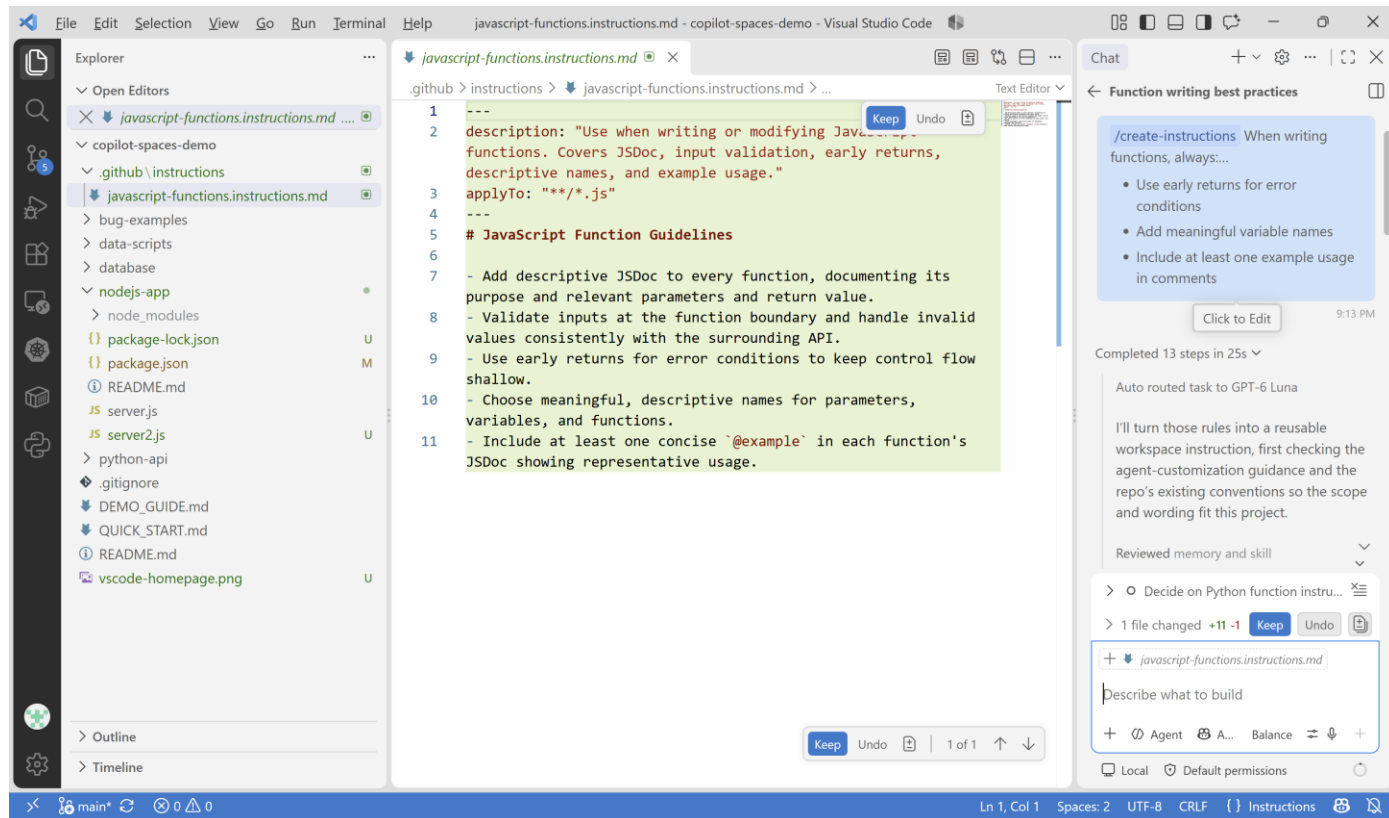


# VS Code customization commands

- Built-in commands automatically create customization folder/file
  - */create-instructions,*
  - */create-skills*
  - */create-agents*
- Configuration commands also provided
  - */instructions,*
  - */skills*
  - */agents*

# /create-instructions example

- Creates `.github/instructions` folder if it doesn't exist
- Creates `functions.instructions.md` file

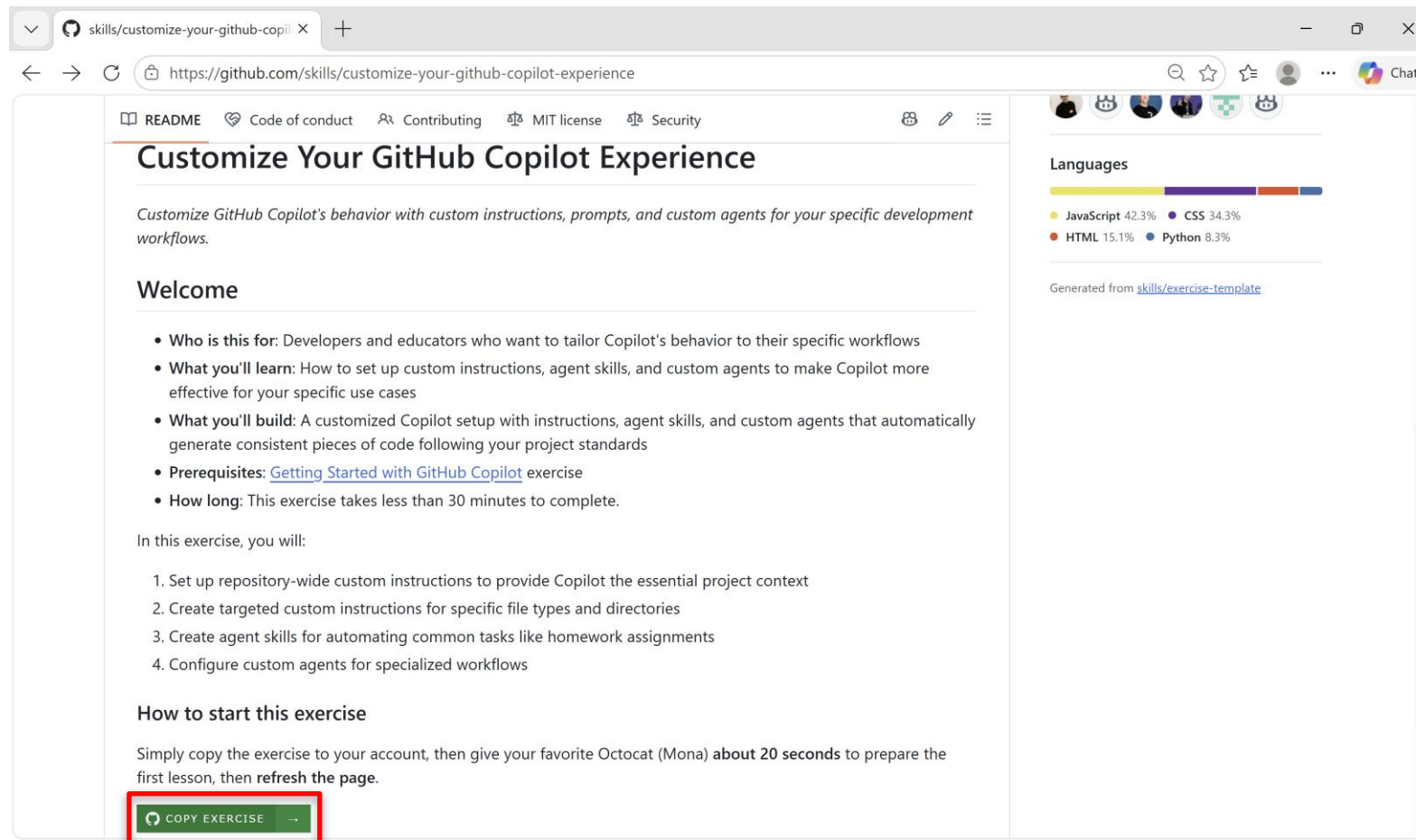


# Labs – GitHub Skills

- “Learn GitHub within GitHub”
  - <https://learn.github.com/skills>
- Collection of self-paced, interactive courses
  - Designed for both beginners and experts
  - Used for course labs
- Covers wide range of GitHub topics
  - GitHub
  - GitHub Copilot
  - GitHub Actions
  - Code security and analysis
- Lab 1: Customize your GitHub Copilot Experience
  - <https://github.com/skills/customize-your-github-copilot-experience>

# Lab – Copy Exercise

- Scroll down and click *Copy Exercise* button



The screenshot shows a web browser window displaying the GitHub repository page for 'Customize Your GitHub Copilot Experience'. The page title is 'Customize Your GitHub Copilot Experience' and the description is 'Customize GitHub Copilot's behavior with custom instructions, prompts, and custom agents for your specific development workflows.' The page includes a 'Welcome' section with a list of bullet points: 'Who is this for', 'What you'll learn', 'What you'll build', 'Prerequisites', and 'How long'. Below this is a section titled 'In this exercise, you will:' followed by a numbered list of four steps. At the bottom, there is a section titled 'How to start this exercise' with instructions to copy the exercise to the account and refresh the page. A red box highlights the 'COPY EXERCISE' button at the bottom left of the page.

skills/customize-your-github-copilot X +

https://github.com/skills/customize-your-github-copilot-experience

README Code of conduct Contributing MIT license Security

## Customize Your GitHub Copilot Experience

Customize GitHub Copilot's behavior with custom instructions, prompts, and custom agents for your specific development workflows.

### Welcome

- **Who is this for:** Developers and educators who want to tailor Copilot's behavior to their specific workflows
- **What you'll learn:** How to set up custom instructions, agent skills, and custom agents to make Copilot more effective for your specific use cases
- **What you'll build:** A customized Copilot setup with instructions, agent skills, and custom agents that automatically generate consistent pieces of code following your project standards
- **Prerequisites:** [Getting Started with GitHub Copilot](#) exercise
- **How long:** This exercise takes less than 30 minutes to complete.

In this exercise, you will:

1. Set up repository-wide custom instructions to provide Copilot the essential project context
2. Create targeted custom instructions for specific file types and directories
3. Create agent skills for automating common tasks like homework assignments
4. Configure custom agents for specialized workflows

### How to start this exercise

Simply copy the exercise to your account, then give your favorite Octocat (Mona) **about 20 seconds** to prepare the first lesson, then **refresh the page**.

**COPY EXERCISE**

### Languages

Generated from [skills/exercise-template](#)

Language	Percentage
JavaScript	42.3%
CSS	34.3%
HTML	15.1%
Python	8.3%

# Lab – Create a new repository

- Login to github.com and click *Create repository*

New repository

https://github.com/new?template\_owner=skills&template\_name=customize-your-github-copilot-experience&owner=%40me&name=skills-cu...

### 1 General

Owner \* ronaldsumida / Repository name \* my-skills-customize-your-github-copilot-experience

my-skills-customize-your-github-copilot-experience is available.

Great repository names are short and memorable. How about [ubiquitous-computing-machine](#)?

Description

Exercise: Customize Your GitHub Copilot Experience

50 / 350 characters

### 2 Configuration

Choose visibility \* Public

Choose who can see and commit to this repository

Connect GitHub Apps 1 app selected

Connect this repository to apps ronaldsumida is subscribed to

### 3 Jumpstart your project with Copilot (optional)

Tell [Copilot cloud agent](#) what you want to build in this repository. After creation, Copilot will open a pull request with generated files - such as a basic app, starter code, or other features you describe - then request your review when it's ready.

Prompt

Describe what you want Copilot to build

0 / 30000 characters

Create repository

# Lab – Go to exercise

- Click *Go to exercise*

The screenshot shows a web browser window displaying a GitHub repository. The browser's address bar shows the URL: `https://github.com/ronaldsumida/my-skills-customize-your-github-copilot-experience`. The repository page is titled "Exercise: Customize Your GitHub Copilot Experience".

On the left, a table lists the repository's files and folders, their commit history, and the time since the last commit:

File/Folder	Commit Message	Commit Hash	Time Ago
github-actions[bot]	Update assignment due dates for exercise start	83b4d85	2 minutes ago
.devcontainer	Initial commit		2 minutes ago
.github	Start exercise		2 minutes ago
assets	Initial commit		2 minutes ago
assignments	Initial commit		2 minutes ago
templates	Initial commit		2 minutes ago
.gitignore	Initial commit		2 minutes ago
LICENSE	Initial commit		2 minutes ago
README.md	Start exercise		2 minutes ago
config.json	Update assignment due dates for exercise start		2 minutes ago
index.html	Initial commit		2 minutes ago

Below the table, the README section is visible, titled "Customize Your GitHub Copilot Experience". It contains a greeting: "Hey ronaldsumida! Monna here. I'm done preparing your exercise. Hope you enjoy! ❤️". Below this, it says: "Remember, it's self-paced so feel free to take a break! 🍌". At the bottom of the README, there is a green button with the text "GO TO EXERCISE" and a right-pointing arrow. This button is highlighted with a red rectangular box.

On the right side of the repository page, there are sections for "Releases", "Packages", "Contributors", and "Languages". The "Languages" section shows a bar chart with the following data:

Language	Percentage
JavaScript	42.3%
CSS	34.3%
HTML	15.1%
Python	8.3%

# Lab – Open in GitHub Codespaces

- Click *Open in GitHub Codespaces*

The screenshot shows a web browser window with the address bar displaying `https://github.com/ronaldsumida/my-skills-customize-your-github-copilot-experience/issues/1`. The page title is "Exercise: Customize Your GitHub Copilot Experience #1". A green "Open" button is visible at the top left of the issue content.

**Theory: What are repository custom instructions?**

Repository custom instructions let you provide Copilot with repository-specific guidance and preferences that help it understand your project context and standards. By creating a `.github/copilot-instructions.md` file, you can ensure that Copilot's suggestions consistently follow your project conventions and coding standards.

The complete set of instructions will be automatically added to all requests that you submit to Copilot Chat in your repository.

**Tip**

Keep instructions short and focused on the "how" of the project. This could be purpose, folder structure, coding standards, key tools, expected formats, etc.

See the [GitHub Docs: Repository Custom Instructions](#) page for more information.

**Activity: Explore the Educational Website Project**

To work with custom instructions, let's first set up our development environment and explore the project structure.

1. Right-click the below button to open the **Create Codespace** page in a new tab. Use the default configuration.

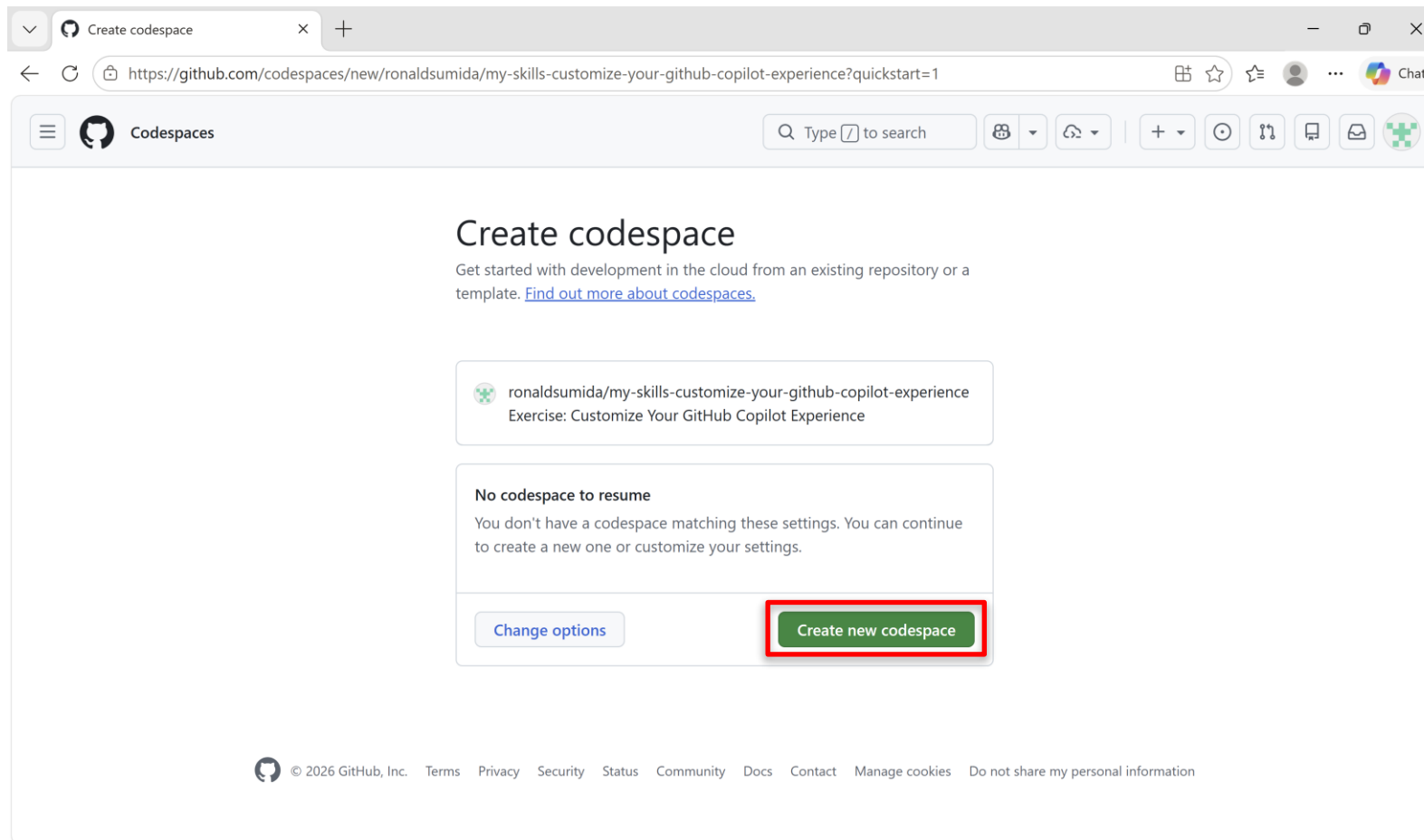
A button labeled "Open in GitHub Codespaces" is highlighted with a red rectangular box.

2. Confirm the **Repository** field is your copy of the exercise, not the original, then click the green **Create Codespace** button.
  - o ☒ Your copy: `/ronaldsumida/my-skills-customize-your-github-copilot-experience`
  - o ☐ Original: `/skills/customize-your-github-copilot-experience`
3. Wait a moment for Visual Studio Code to load in your browser and for all extensions to install.
  - o Ensure the **Live Preview** extension is activated.
4. Right-click on `index.html` and select **Show Preview** to see the website in action.

The right sidebar of the GitHub interface shows various settings for the issue, including Labels, Projects, Milestone, Relationships, Development, Notifications, and Participants.

# Lab – Create Codespace

- Click *Create new codespace*

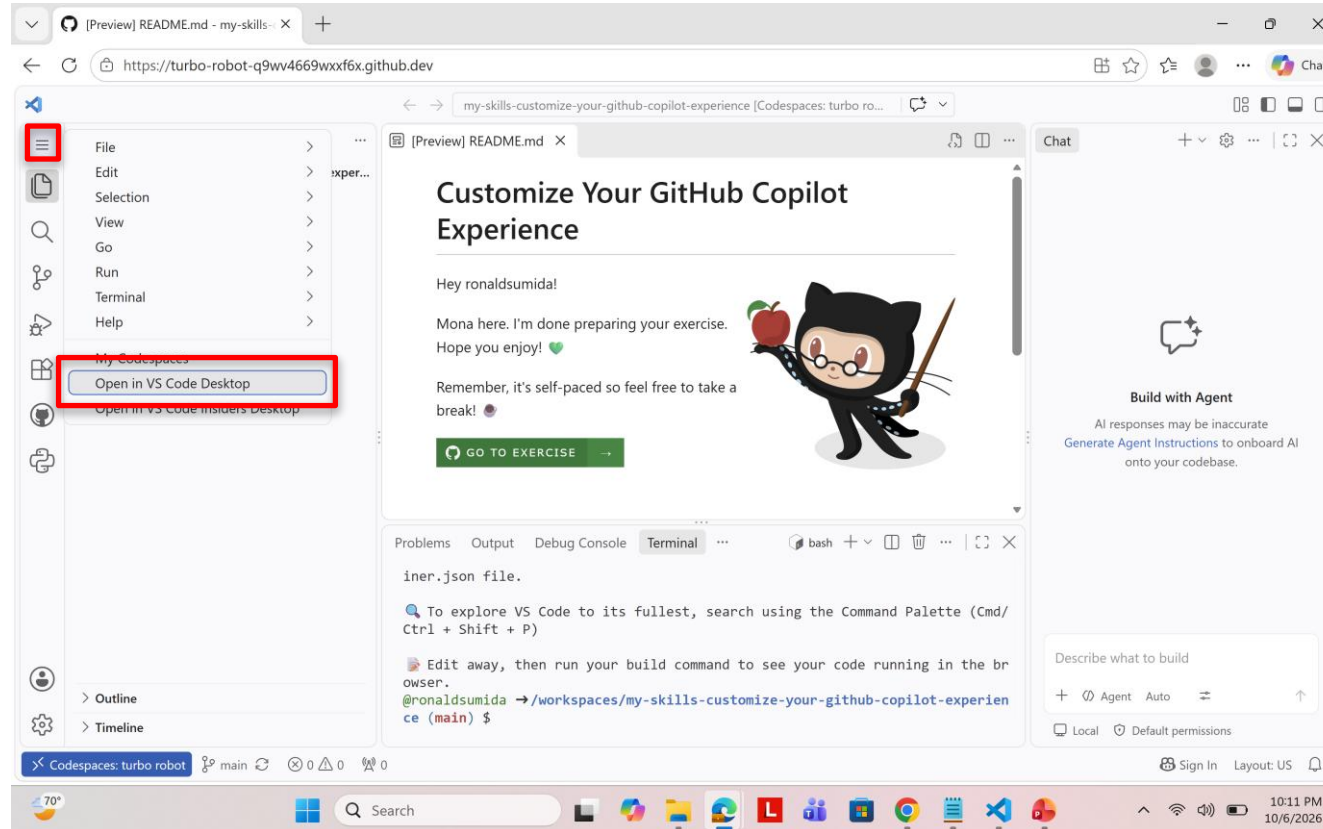


# GitHub Codespaces

- Cloud-powered development environment
  - Native to GitHub
  - Fully configured
  - Secure
  - Used for all of the course labs
- Client access
  - Browser
    - Connect through browser-based editor
      - May not work in Incognito mode or with ad blockers enabled
    - Use lightweight VS Code interface
  - VS Code
    - Connect through GitHub Codespaces extension
    - Use local interface

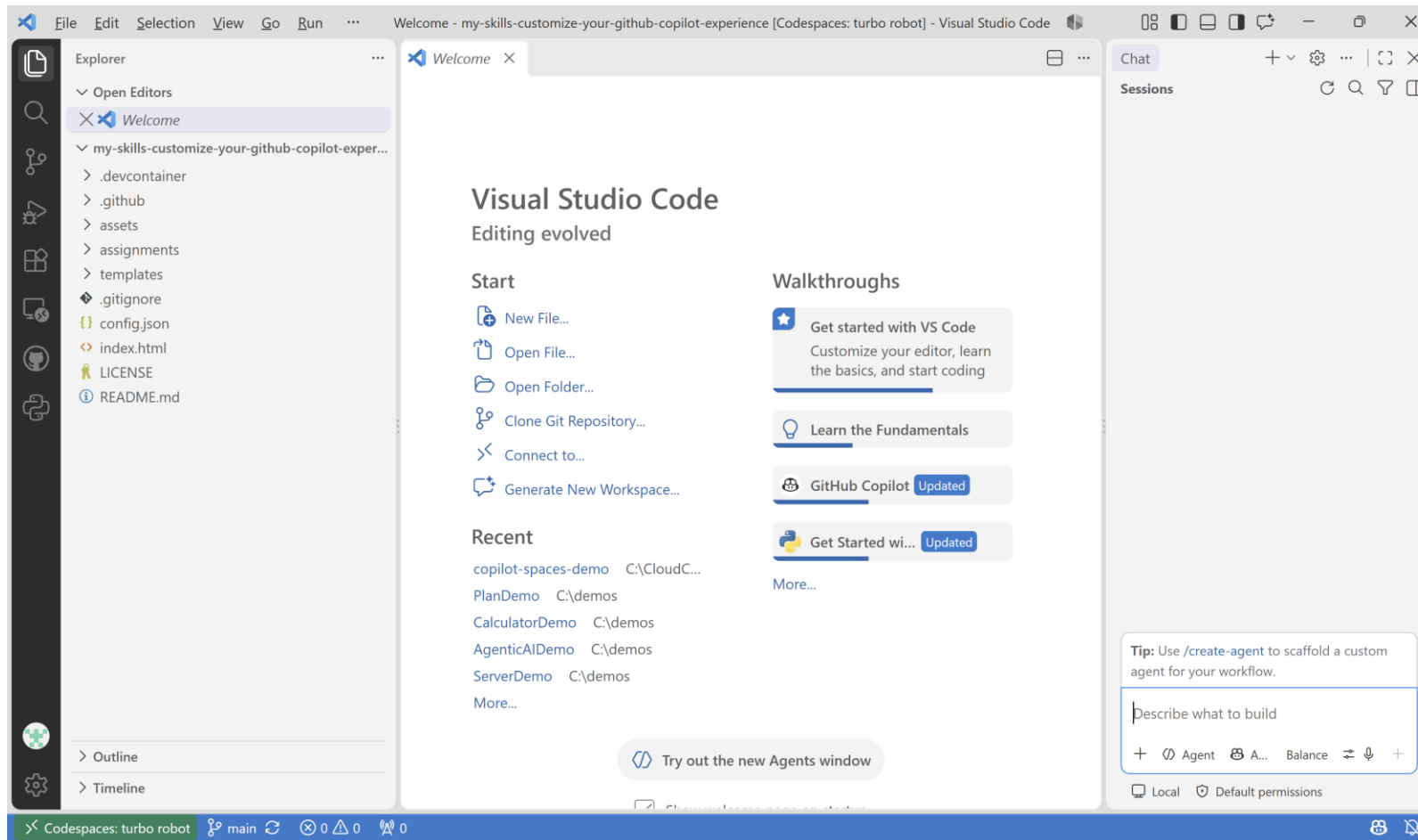
# GitHub Codespace in browser

- Opens development environment in browser
  - Select *Open VS Code Desktop* to transfer to VS Code desktop



# GitHub Codespace in VS Code desktop

- More convenient way to work with GitHub Codespaces



# Summary and conclusions

- GitHub Copilot provides wide range of customization options
  - Instructions
    - Define baseline coding standards and guidelines
  - Agent skills
    - Package reusable, step-by-step workflows, scripts and recipes for specific tasks
  - Custom agents
    - Act as dedicated specialist persona with distinct rule, toolset and runtime
- Conclusion
  - Learning how to prompt GitHub Copilot is just the starting point
  - Learning how to customize GitHub Copilot is the key to using it effectively